



Cifrado César

Facultad de Ingeniería
Criptografía

Dr. Alfonso Francisco de Abiega l'Eglise
2027-1

Grupo: 3

Ayala Hernández María Fernanda
López Hernández Miriam Amisadai
Tepal Briseño Hansel Yael
Ugartechea González Luis Antonio

1 Explicación del Cifrado César

1.1 Declaración de Variables

```
1 char texto[100];
2 char descifrado[100];
3 int desplazamiento, valido = 1;
4 int manual = 0;
```

Listing 1: Declaración de variables y arreglos de caracteres.

Se declaran los arreglos de caracteres con un límite de 100 espacios para almacenar el mensaje cifrado y el texto descifrado, así como las variables enteras para la llave de cifrado (desplazamiento) y banderas lógicas de control (valido y manual).

1.2 Selección de la Llave

```
1 printf("Desea ingresar manualmente la llave de cifrado? (1 para
   sí, 0 para no): ");
2 if (scanf("%d", &manual) != 1) {
3     printf("No se pudo leer la opción.\n");
4     return 1;
5 }
6 if (manual) {
7     printf("Ingrese la llave de cifrado (0-25): ");
8     if (scanf("%d", &desplazamiento) != 1) {
9         printf("No se pudo leer la llave.\n");
10        return 1;
11    }
12    if (desplazamiento < 0 || desplazamiento > 25) {
13        printf("La llave debe estar entre 0 y 25.\n");
14        return 1;
15    }
16 } else {
17     srand((unsigned int)time(NULL));
18     desplazamiento = rand() % 26;
19 }
```

Listing 2: Selección de la llave de cifrado.

El uso del formato %99[^\n] en scanf nos permite leer hasta 99 caracteres (dejando espacio para el carácter nulo) incluyendo espacios, deteniéndose al encontrar un salto de línea. Posteriormente, se pregunta si la llave se ingresará de manera manual. Si el usuario elige que sí, se lee la llave y se valida que esté en el rango del alfabeto (0 a 25). Si no, se utiliza srand con el tiempo del sistema para inicializar la semilla y se genera un desplazamiento aleatorio usando la operación módulo 26.

Es importante destacar el uso de (`unsigned int`) al llamar a `srand`. La función `time(NULL)` devuelve un valor de tipo `time_t` (que puede tener signo), y al convertirlo a un entero sin signo, garantizamos que la semilla alimentada al generador de números sea estrictamente positiva. Esto nos da una correcta generación de `rand()` y es fundamental al aplicar el módulo (`% 26`), ya que en C, el operador módulo aplicado sobre números negativos produce un resultado negativo. Al forzar valores estrictamente positivos, evitamos desplazamientos negativos que romperían la lógica de recorrido del alfabeto.

Para evitar un cifrado erróneo de símbolos no contemplados, se realiza una iteración sobre la cadena para verificar que todos los caracteres pertenezcan al alfabeto inglés (mayúsculas 'A'-'Z', minúsculas 'a'-'z'). Si se encuentra un carácter que no cumple estas condiciones, la bandera `valido` se marca como falsa (0), interrumpiendo el bucle y abortando la ejecución con un mensaje de error.

1.3 Proceso de Cifrado

```
1  for (int i = 0; texto[i] != '\0'; i++) {
2      if (texto[i] >= 'A' && texto[i] <= 'Z') {
3          texto[i] = (char)('A' + (texto[i] - 'A' + desplazamiento
4              ) % 26);
5      } else if (texto[i] >= 'a' && texto[i] <= 'z') {
6          texto[i] = (char)('a' + (texto[i] - 'a' + desplazamiento
7              ) % 26);
8      }
9  }
```

Listing 3: Proceso de cifrado del mensaje.

Este fragmento es el la matemática del algoritmo César. Se evalúa cada carácter válido: si es letra, se aplica la fórmula modular (`caracter - base + desplazamiento`) `% 26`. Restar la base ('A' o 'a') mapea la letra al rango numérico de 0 a 25. Luego se suma el desplazamiento y se aplica el módulo 26 para que el conteo regrese a cero si sobrepasa la letra 'Z'. Finalmente, se suma la base ASCII para recuperar el carácter cifrado correspondiente. Los espacios son ignorados intencionalmente y no sufren mutación.

1.4 Proceso de Descifrado

```
1  for (int i = 0; descifrado[i] != '\0'; i++) {
2      if (descifrado[i] >= 'A' && descifrado[i] <= 'Z') {
3          descifrado[i] = (char)('A' + (descifrado[i] - 'A' -
4              desplazamiento + 26) % 26);
5      } else if (descifrado[i] >= 'a' && descifrado[i] <= 'z') {
6          descifrado[i] = (char)('a' + (descifrado[i] - 'a' -
7              desplazamiento + 26) % 26);
8      }
9  }
```

Listing 4: Proceso de descifrado del mensaje.

Primero, el mensaje que ha sido cifrado en el propio arreglo `texto` se copia al arreglo `descifrado`, cuidando de copiar progresivamente el carácter nulo `'\0'` para delimitar correctamente el string. El proceso de descifrado es equivalente al cifrado pero a la inversa: se resta el desplazamiento. Como en el lenguaje C la operación módulo de números negativos puede arrojar un resultado negativo, se le suma 26 previamente, garantizando que `%` 26 siempre devuelva un índice positivo dentro del alfabeto.

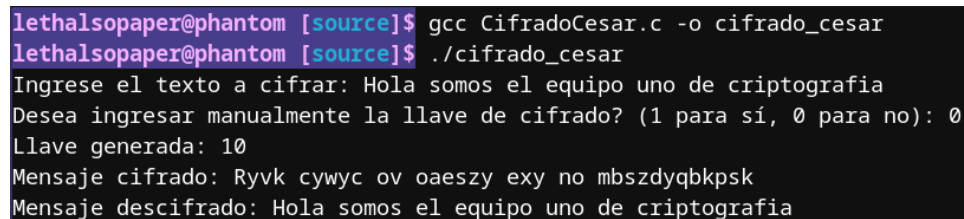
1.5 Salida de Resultados

```
1 printf("Llave generada: %d\n", desplazamiento);
2 printf("Mensaje cifrado: %s\n", texto);
3 printf("Mensaje descifrado: %s\n", descifrado);
4
5 return 0;
```

Listing 5: Salida de resultados del programa.

El programa concluye imprimiendo en pantalla la llave numérica utilizada (lo cual es vital si fue generada mediante el modo aleatorio), la cadena resultante del cifrado y la cadena tras someterse al proceso de descifrado.

1.6 Salida en terminal



```
lethalsopaper@phantom [source]$ gcc CifradoCesar.c -o cifrado_cesar
lethalsopaper@phantom [source]$ ./cifrado_cesar
Ingrese el texto a cifrar: Hola somos el equipo uno de criptografia
Desea ingresar manualmente la llave de cifrado? (1 para sí, 0 para no): 0
Llave generada: 10
Mensaje cifrado: Ryvk cywyc ov oaeszy exy no mbszdyqbkpsk
Mensaje descifrado: Hola somos el equipo uno de criptografia
```

Figure 1: Ejemplo de salida en terminal.

1.7 Repositorio y Código Fuente

Código fuente:

Link del repositorio: <https://github.com/HansMarcus01/Criptografia-2027-1>